COMP4322 Internetworking Protocols, Software and Management

Group Project Report

LSR Protocol Simulator in Java

YANG Xikun

YANG Jinkun

Note: Our project is open-source. Please refer to https://github.com/FrankYang0610/link-state-routing

I. Summary of the LSR algorithm

1.1 Overview of the LSR algorithm

Link State Routing (LSR) is a routing algorithm used in many protocols, such as OSPF and IS-IS. In simple terms, LSR allows each router to first "learn the entire network topology" and then independently compute the shortest paths. Therefore, the key to its implementation is that each node maintains a complete map of the network topology and then uses a shortest path algorithm to build its routing table.

Unlike Distance Vector routing (where each router only knows how to reach destinations through its neighbors), in LSR every router has a view of the "entire map."

In practical networking, routers in LSR discover directly connected neighbors through Hello packets. Each link is then assigned a cost (such as delay or bandwidth). After that, every router advertises its neighbors and the cost of each link through a process called flooding. Finally, routers use Dijkstra's shortest path algorithm to compute the shortest paths to all nodes, and each router generates its own routing table.

The advantages of LSR include fast convergence, more accurate routing, and a lower likelihood of routing loops. However, its disadvantages are also clear: it requires more memory, involves higher computational complexity, and the flooding process incurs significant overhead.

1.2 Running steps of the LSR algorithms

In this project, the link-state advertisements are assumed to be available from an input `.lsa` file that describes the graph. Therefore, our implementation focuses on the shortest-path-first computation stage of LSR, where the source router builds the topology graph and runs Dijkstra's algorithm to get the routing table. Specifically, the pseudocode of our algorithm is as follows:

```
LINK-STATE-ROUTING(G = (V, E), s)
    INITIALIZE-SINGLE-SOURCE(G, s)
    S ← ∅
    Q ← V
    while Q ≠ ∅
        u ← EXTRACT-MIN(Q)
        S ← S ∪ {u}
        for each v ∈ adj[u]
            if v.d > u.d + w(u, v) then
                v.d ← u.d + w(u, v)
                v.π ← u
    return BUILD-ROUTING-TABLE(G, s)

INITIALIZE-SINGLE-SOURCE(G = (V, E), s)
    for each v ∈ V[G]
        v.d ← ∞
        v.π ← NIL
    s.d ← 0

BUILD-ROUTING-TABLE(G = (V, E), s)
    for each v ∈ V – {s}
        next[v] ← FIRST-HOP(s, v)
        cost[v] ← v.d
    return (next, cost)

FIRST-HOP(s, v)
    if v.π = NIL then
        return NIL
    u ← v
    while u.π ≠ s
        u ← u.π
        if u.π = NIL
            return NIL
    return u
```

Of course, in actual implementation, we also store the information from each iteration of Dijkstra's algorithm to support the "SS" (Single Step) mode for user presentation. These parts are omitted here.

1.3 Step-by-step Illustration of the Dijkstra's Algorithm

Next, we demonstrate how Dijkstra's algorithm works. We use the figure from the well-known textbook ***Introduction to Algorithms***, which clearly illustrates how the algorithm finds shortest paths starting from a source node *s*.

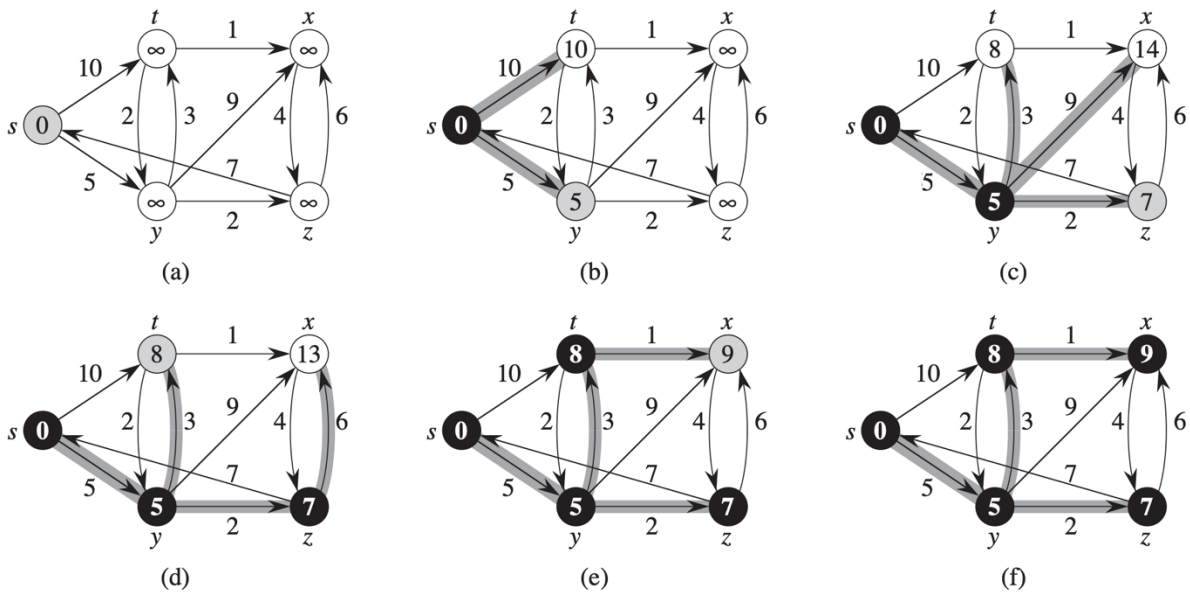


Fig 1.1. The execution of Dijkstra's algorithm. The shortest-path costs are shown inside each node, and shaded edges show where each node comes from on the shortest path. Black nodes belong to set *S*, while white nodes are in the min-priority queue *Q*. **(a)** The state before the first iteration of the while loop. The shaded node has the smallest *d* value and is selected as *u*. **(b)-(f)** The state after each iteration. In each case, the shaded node is chosen as *u* for the next iteration. The *d* values and predecessors in (f) are the final results.

1.4 LSR Reference

[1] "Link-state routing protocol," *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/wiki/Link-state_routing_protocol. [Accessed: Apr. 16, 2026].

[2] "Distance-vector routing protocol," *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/wiki/Distance-vector_routing_protocol. [Accessed: Apr. 17, 2026].

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[4] "Dijkstra's algorithm," *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm. [Accessed: Apr. 16, 2026].

II. Summary of our design and implementation of the LSR protocol

2.1 General architecture design

We used Java and classic object-oriented programming (OOP) principles to design our program. We also developed a GUI and implemented a separation between the GUI and the business logic. Please refer to **Section 2.2** first.

Next, we present several representative screenshots of our program.

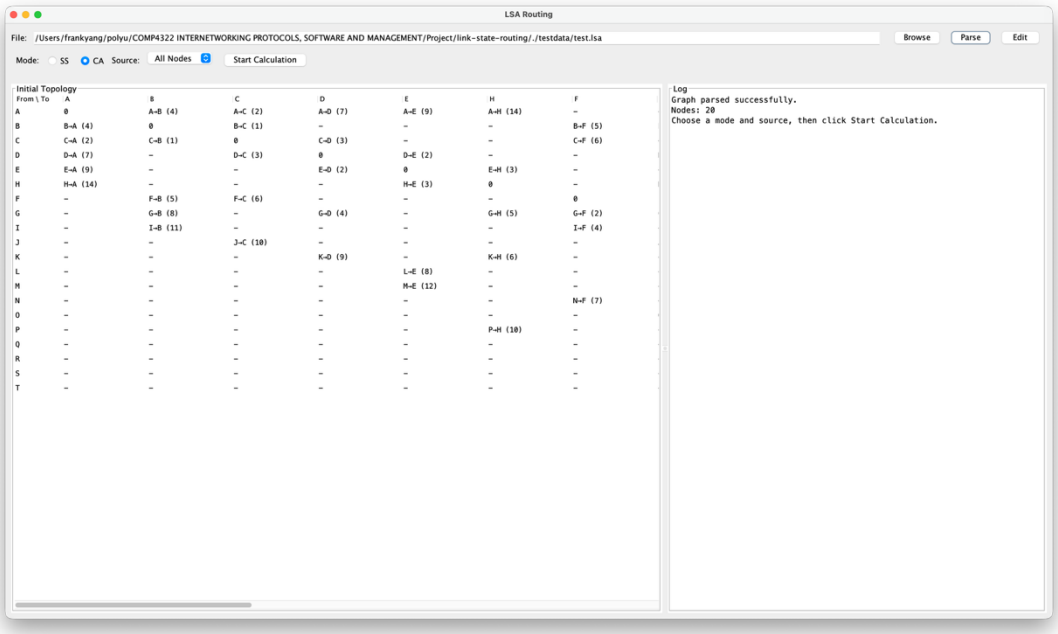


Fig 2.1. Main interface after parsing the .lsa file.

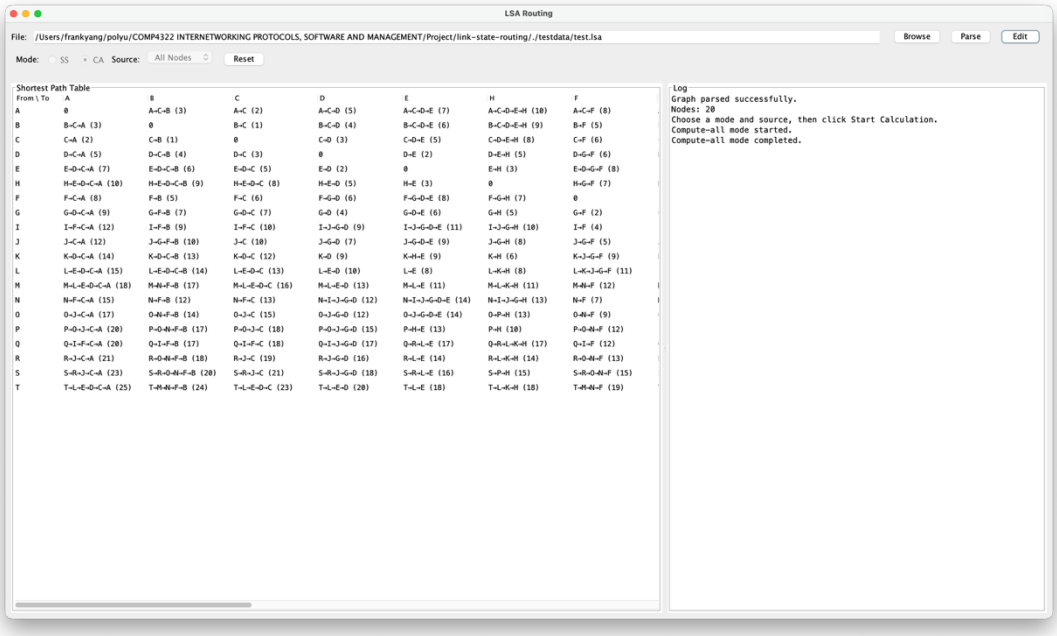


Fig 2.2. All-pairs shortest routes and total costs displayed after running in CA mode.

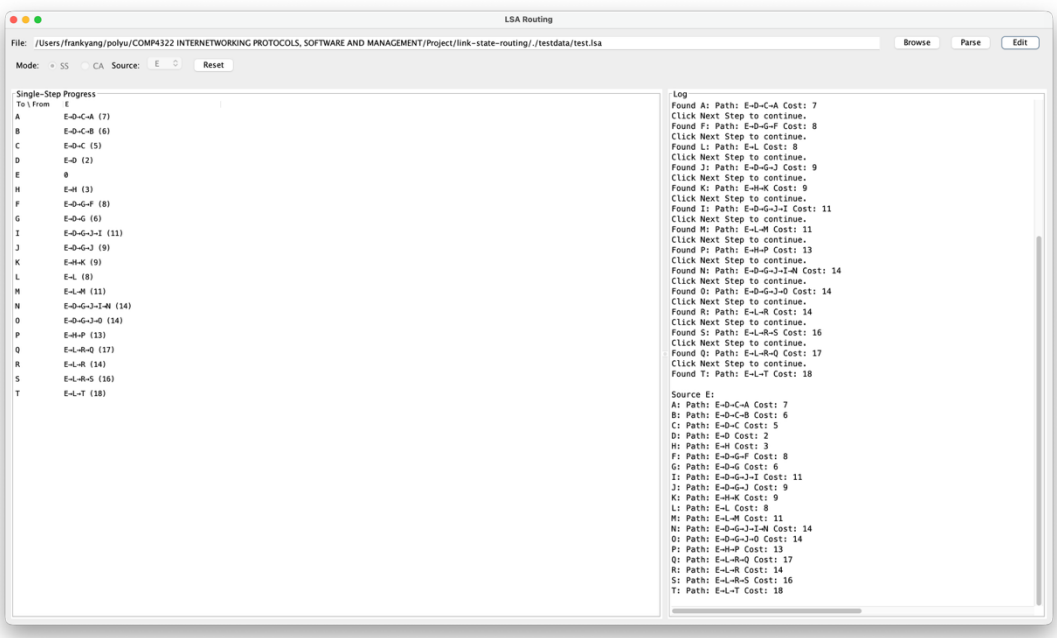


Fig 2.3. Shortest routes and total costs from source node E to all other nodes after running in SS mode.

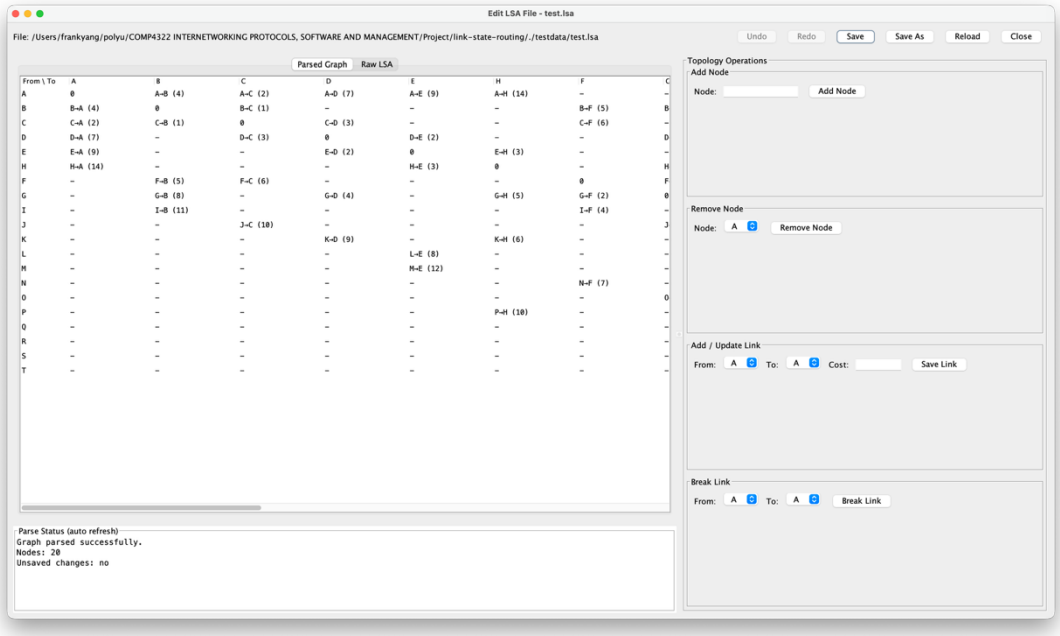


Fig 2.4. Graph Topology Editing Page. You can edit the graph in the Topology Operations panel.

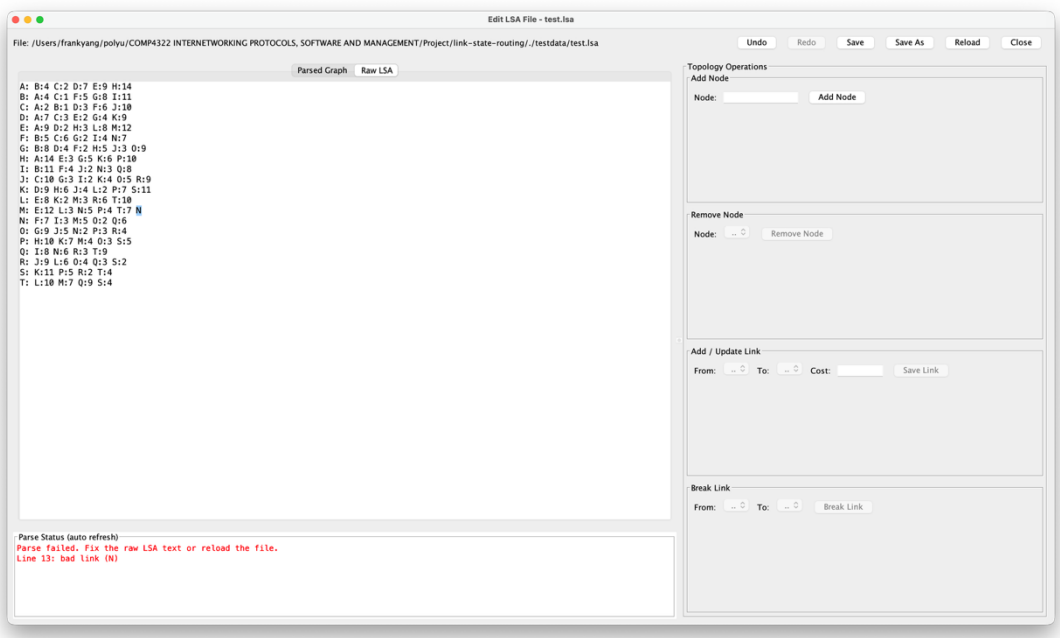


Fig 2.5. Direct `.lsa` file editing page. Note the parse status message in the lower-left corner indicating a graph error. This is because the last token on node *M*'s line is a newly added N without a corresponding cost. Even if a cost were provided, it would still trigger an error, since there is already a link from *M* to *N* with a cost of 5.

2.2 Class list, responsibilities and functionalities

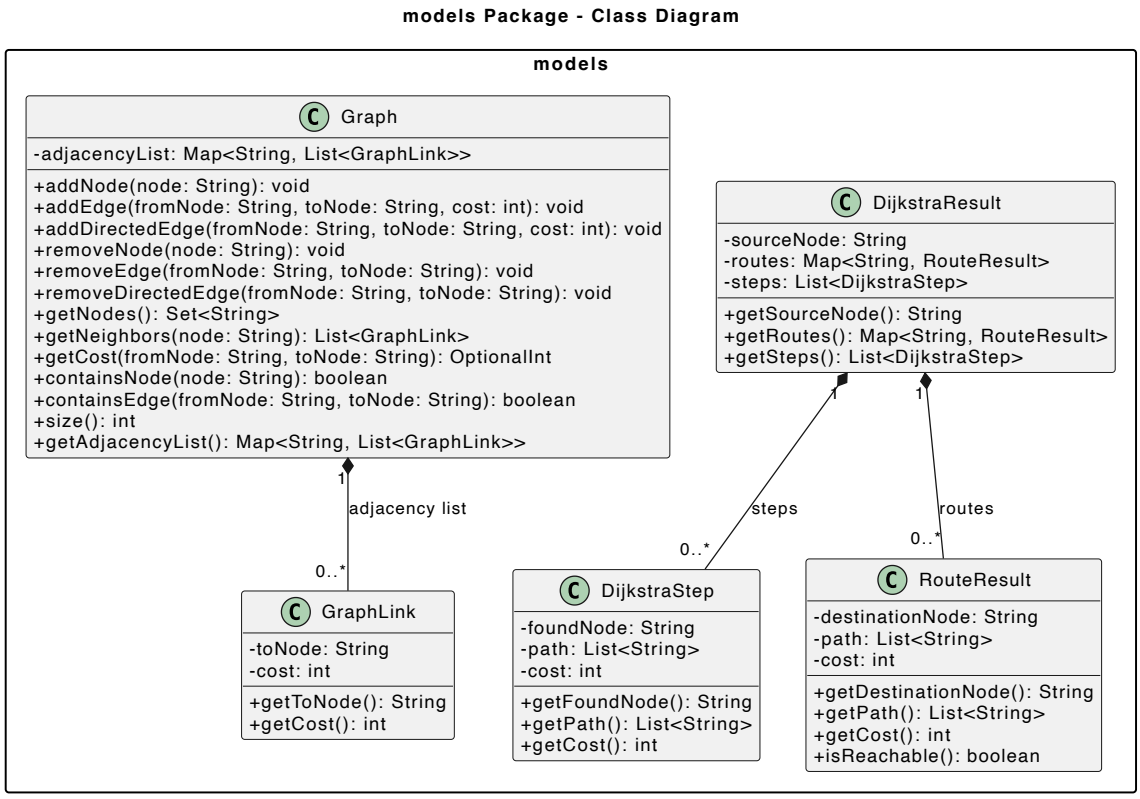
- **`LSRCompute`:** This is the program's entry point. If no command-line arguments are provided, the GUI is launched. If arguments are provided, the program runs in CLI mode as specified in the instruction document. For example, running `java LSRCompute <file.lsa> <source> [SS|CA]` will output the results directly in the terminal.
- **`LSRGUI`:** The main graphical user interface class, built with Javax Swing. It allows user to select a `.lsa` file, parse it, and then display the network topology. The user can also select the source node and SS/CA mode, then computation logs along with step-by-step execution results will be presented.
- **`LSREditorGUI`:** The GUI topology editor class. It allows users to add/remove nodes, create/update links, and disconnect links through a GUI, as well as directly edit the raw LSA text and save the changes back to the file.
- **`LSRFileParser`:** The LSA file parser. It converts a `.lsa` file or raw text into a `Graph` object. It does not check for graph format errors.
- **`LSRFileWriter`:** The LSA file writer. It serializes the in-memory `Graph` topology back into `.lsa` format for saving the edited network.
- **`LSRGraphValidator`:** The graph validator. It checks whether the graph is valid, for example, whether reverse links match, whether bidirectional link costs are consistent, and whether there are invalid nodes or duplicate links.
- **`LSRDijkstraCalculator`:** The Dijkstra shortest-path computation class. Given a source node, it calculates the shortest paths to all other nodes and records each node discovery step required for visualization in SS mode.
- **`LSRResultFormatter`:** The result formatting class. It converts the output of the Dijkstra algorithm into CLI/GUI-friendly text, including step-by-step outputs and the final summary table.
- **`LSRTopologyService`:** The topology operations service class. It provides editor functions such as adding and removing nodes, updating links, and disconnecting links, and avoids mutating the original graph by operating on copied graph instances.
- **`models`:** The data model package containing the core data structures. It does not handle the UI or file I/O. Instead, it defines what the network graph is and how computation results are represented.
    - **`Graph`:** The graph model, using an adjacency list to store nodes and links (edges). It provides basic graph operations including adding/removing nodes, adding/removing edges, and querying neighbors and corresponding costs.

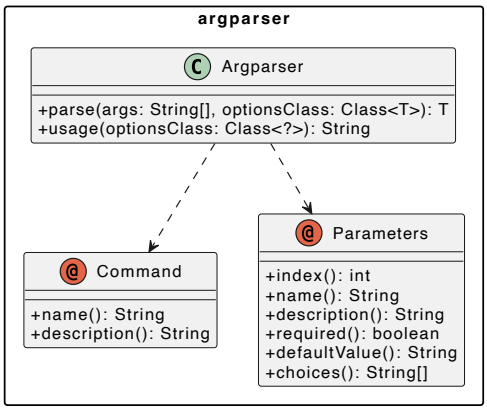
- **`GraphLink`:** The model for a graph link (edge). It stores the target node's name and the link cost.
- **`DijkstraStep`:** The record of a single step in the Dijkstra algorithm. Each instance represents a newly finalized node at that step, along with the path from the source node to it and the current shortest cost.
- **`DijkstraRouteResult`:** The Dijkstra's result for a single destination. It stores the destination node, the full path, and the total cost, and can determine whether the destination is reachable.
- **`DijkstraResult`:** The result of a single Dijkstra computation (from a source node to all other nodes). It stores the source node, the routes to each destination node (refer to `DijkstraRouteResult`), and the list of steps for SS-mode visualization.

- **`argparser`:** A command-line argument parsing package (like Python's `argparse`). It uses annotations to define commands and parameters, and then uses reflection to convert `String[] args` into a parameter list object.
  - **`Argparser`:** Argument parsing class. It reads the `@Parameters` annotations, handles required arguments, default values, and allowed option ranges, and generates usage text.
  - **`Command`:** Annotation for command classes. It is used to mark a parameter configuration class with its corresponding command name and description, for example enabling the usage output to display something like `java LSRCompute` ....
  - **`Parameters`:** A field-level annotation for command-line arguments. It specifies an argument's position, name, whether it is required, its default value, and the set of allowed values (e.g., `SS|CA`).

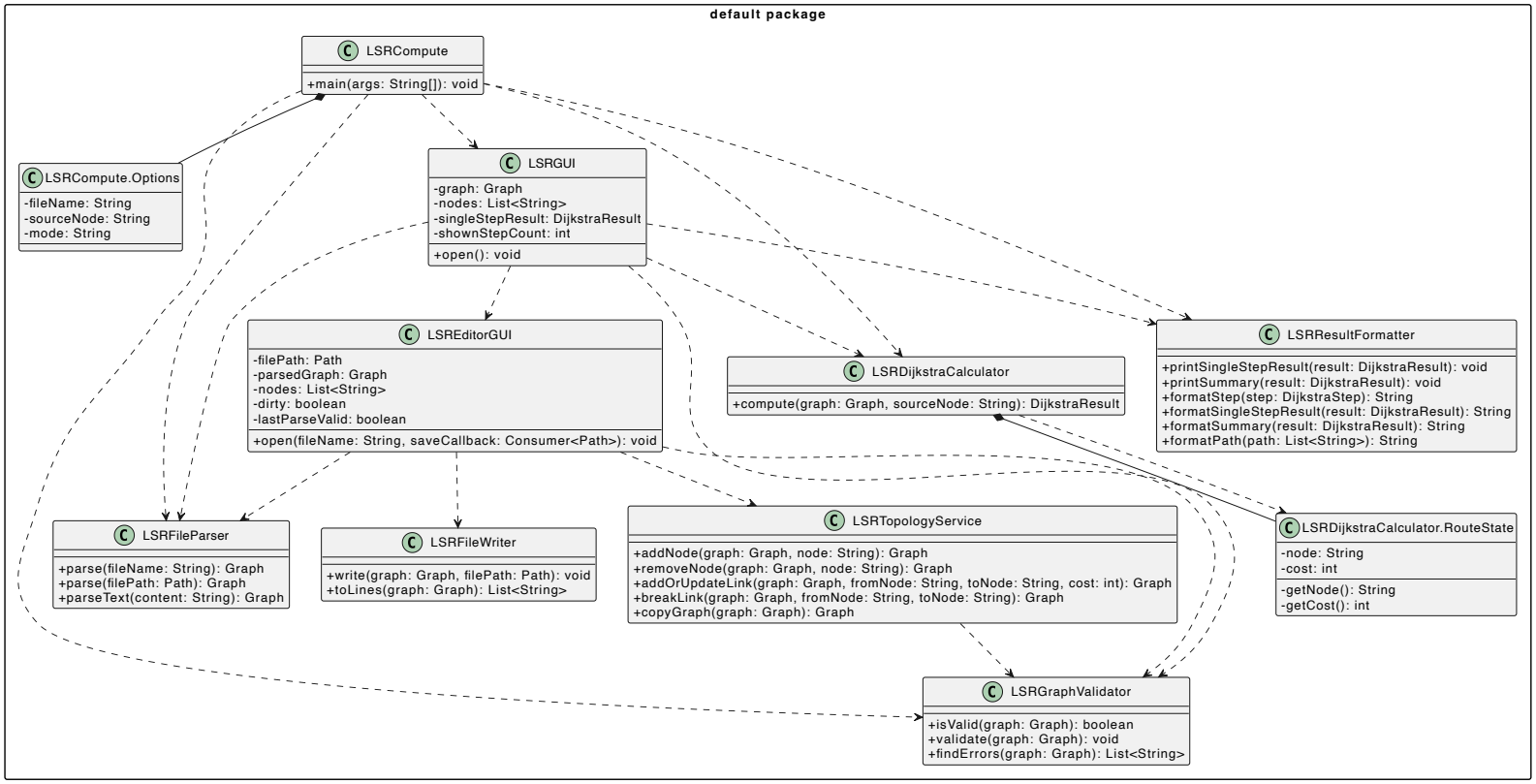
2.3 UML diagram and system architecture diagram

We first present the UML diagrams and dependencies for the `models` and `argparser` packages, which define the core data structures of the system. We then present the UML diagram and dependencies for the default package, which contains the core logic and overall system architecture.





Default Package - Class Diagram



2.4 Test and test results

All business classes have been thoroughly unit tested using the JUnit 5 framework, and **all tests have passed**. The JUnit 5 framework is included in the `lib/` directory and the relevant build files have already been configured, so you can open our code root directory directly on your computer without needing to separately set up Maven or download JUnit 5.

For the GUI component, we used real-user testing. All related functions and program behaviors were verified to be correct, and the error reporting feature is also fully functional.

All unit tests we implemented and passed are listed in the table below.

Unit Test Class	Test Case	What It Checks	Result
`GraphAndLinkTest`	`newGraphStartsEmpty`	Checks that a new Graph starts with no nodes.	Passed
`GraphAndLinkTest`	`addNodeIsIdempotent`	Checks that adding the same node more than once does not create duplicates.	Passed
`GraphAndLinkTest`	`addDirectedEdgeCreatesOnlyOneDirection`	Checks that a directed edge only creates one-way connectivity.	Passed
`GraphAndLinkTest`	`addEdgeCreatesTwoDirectedLinks`	Checks that an undirected edge creates links in both directions.	Passed
`GraphAndLinkTest`	`addingSameEdgeUpdatesExistingCost`	Checks that adding an existing edge again updates its cost instead of duplicating it.	Passed
`GraphAndLinkTest`	`removeDirectedEdgeRemovesOnlyRequestedDirection`	Checks that removing a directed edge does not remove the reverse edge.	Passed
`GraphAndLinkTest`	`removeEdgeRemovesBothDirections`	Checks that removing an undirected edge removes both directions.	Passed
`GraphAndLinkTest`	`removeNodeRemovesIncidentLinks`	Checks that removing a node also removes links pointing to that node.	Passed
`GraphAndLinkTest`	`graphViewsAreReadOnly`	Checks that returned graph collections cannot be modified directly by outside code.	Passed
`GraphAndLinkTest`	`graphLinkRejectsNegativeCost`	Checks that a link with negative cost is rejected.	Passed
`DijkstraAndRouteTest`	`computesShortestRoutesFromSource`	Checks that Dijkstra correctly computes shortest paths from the source node.	Passed
`DijkstraAndRouteTest`	`recordsSingleStepDiscoveryOrder`	Checks that SS mode records discovered nodes in the correct order.	Passed
`DijkstraAndRouteTest`	`reportsUnreachableRoutes`	Checks that unreachable nodes are reported correctly.	Passed

Unit Test Class	Test Case	What It Checks	Result
`DijkstraAndRouteTest`	`rejectsInvalidComputeInputs`	Checks that invalid Dijkstra inputs, such as null graph or unknown source, are rejected.	Passed
`DijkstraAndRouteTest`	`resultObjectsCopyAndProtectTheirCollections`	Checks that result objects protect their internal lists and maps from outside modification.	Passed
`DijkstraAndRouteTest`	`routeResultReachabilityDependsOnPath`	Checks that RouteResult decides reachability based on whether the path is empty.	Passed
`TopologyServiceTest`	`addNodeReturnsNewGraphWithoutChangingOriginal`	Checks that adding a node returns a new graph and does not mutate the original graph.	Passed
`TopologyServiceTest`	`removeNodeRemovesIncidentLinks`	Checks that topology service removes a node and all related links correctly.	Passed
`TopologyServiceTest`	`addOrUpdateLinkKeepsLinksSymmetric`	Checks that adding or updating a link keeps both directions consistent.	Passed
`TopologyServiceTest`	`breakLinkRemovesBothDirections`	Checks that breaking a link removes both directions.	Passed
`LSRProgramTest`	`fileParserReadsValidLsaFile`	Checks that the parser can read a valid .lsa file into a graph.	Passed
`LSRProgramTest`	`fileParserReadsValidLsaText`	Checks that the parser can read valid LSA text directly.	Passed
`LSRProgramTest`	`fileParserRejectsBadInput`	Checks that malformed LSA input is rejected.	Passed
`LSRProgramTest`	`validatorAcceptsValidGraphAndReportsBrokenGraph`	Checks that valid graphs pass validation and broken graphs report clear errors.	Passed
`LSRProgramTest`	`formatterPrintsStepsSummaryAndUnreachableRoutes`	Checks that step output, summary output, paths, and unreachable routes are formatted correctly.	Passed
`LSRProgramTest`	`fileWriterSavesGraphAsParseableLsaFile`	Checks that saved .lsa files can be parsed again correctly.	Passed
`LSRProgramTest`	`argparserParsesDefaultsChoicesAndUsage`	Checks that command-line parsing handles defaults, choices, and usage text correctly.	Passed
`LSRProgramTest`	`lsrComputeRunsComputeAllMode`	Checks that LSRCompute runs CA mode and prints correct shortest path results.	Passed
`LSRProgramTest`	`lsrComputeReportsUnknownSourceAndUsageErrors`	Checks that the program reports unknown source nodes and invalid mode arguments correctly.	Passed

2.5 Coding references

[1] "Object-oriented programming," *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/wiki/Object-oriented_programming. [Accessed: Apr. 17, 2026].

[2] "Projects," *JetBrains IntelliJ IDEA Documentation*. [Online]. Available: https://www.jetbrains.com/help/idea/creating-and-managing-projects.html. [Accessed: Apr. 17, 2026].

Term Project Marking Sheet

Group No: N/A		
Members and contributions:		
Name	**Student ID**	**Contributions**
YANG Xikun	23101291d	Code; Report; Presentation. Equal contribution.
YANG Jinkun	23113164d	Code; Report; Presentation. Equal contribution.
Checklist:		
Summary of the LSR algorithm (10%) - Correctness, easy to read, no grammatical errors or typos, high quality illustrations, full bibliography, etc.		
Summary of your design and implementation of the LSR protocol (20%) - Correctness, thoroughness, easy to read, no grammatical errors or typos, user-friendly interfaces, well-discussed test results, etc.		
Quality of your project's source code (20%) - Good naming and coding convention used in your source code - Correct implementation of the required functionalities of the LSR protocol - Implementation of optional features for the project		
Quality of your demonstration of your project (50%) - Present the project overview clearly - Compile the source code successfully - Execute the program without runtime errors - Successfully demonstrate the program's functionalities and ease of use - Manage the Q&A section smoothly		
Total		